

ARQUITETURA CORPORATIVA

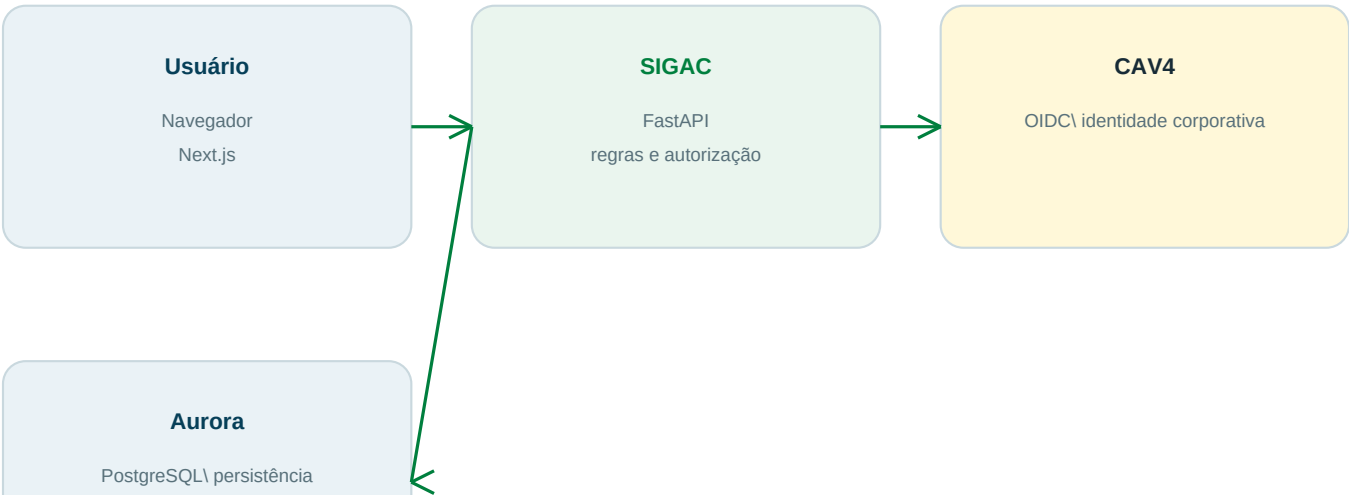
# SIGAC — Arquitetura da Solução

Sistema de Gestão de Acesso ao Armazenamento Científico

Baseline técnico para decisão, construção, operação e governança

| Controle do documento    | Valor                                    |
|--------------------------|------------------------------------------|
| Versão                   | 2.1                                      |
| Status                   | Para validação do gestor e arquitetura   |
| Escopo                   | Primeira fase do SIGAC                   |
| Autoridade de identidade | CAV4 — grupos, papéis e permissões       |
| Persistência             | Aurora PostgreSQL                        |
| Stack                    | Next.js · FastAPI · SQLAlchemy · Alembic |

## Visão executiva



**Resumo executivo.** O SIGAC é uma aplicação web corporativa com frontend Next.js e backend FastAPI organizado em módulos e camadas. O CAV4 é a autoridade de autenticação e identidade; o Aurora PostgreSQL é a fonte persistente do sistema; o acesso às pastas é consultivo na primeira fase, sem upload ou manipulação de arquivos.

# Índice

Placeholder for table of contents

0

# 1. Objetivo e escopo

Este documento descreve a arquitetura implementada e as decisões técnicas do SIGAC, servindo como base para validação do gestor, evolução do produto e comunicação com infraestrutura, segurança e desenvolvimento.

## 1.1. Princípios que governam a solução

- CAV4 é a autoridade de identidade: o SIGAC não inventa, ele interpreta os claims corporativos aprovados.
- Negação por padrão: usuário sem papel reconhecido não recebe acesso por fallback.
- Backend é a autoridade de autorização: o frontend pode ocultar ações, mas nunca concede permissão.
- Contratos estáveis: rotas, endpoints e regras de negócio são preservados durante a evolução visual.
- Observabilidade sem exposição: rastrear operações sem registrar credenciais, tokens ou dados sensíveis.
- Evolução incremental: separar o que está implementado do que depende de CAV4, DBA, infraestrutura ou decisão do gestor.

## 1.2. Leitura recomendada

Gestores podem concentrar-se nas seções 1, 5, 6, 10, 11 e 12. Arquitetura e desenvolvimento devem validar as seções 2, 3, 4 e 7. Segurança e infraestrutura devem validar as seções 5, 9 e 10. O documento não substitui os contratos oficiais do CAV4 nem a aprovação de acesso ao Aurora.

### Escopo vigente da primeira fase:

- Autenticação corporativa pelo CAV4/OIDC.
- Consulta de grupos, roles, valores de informação e dados administrativos do usuário no CAV4.
- Cadastro e gestão de projetos, participantes, status, áreas responsáveis e relatórios conforme endpoints disponíveis.
- Listagem de pastas existentes no servidor corporativo e exibição de permissões de leitura/escrita.
- Auditoria de operações relevantes e observabilidade do backend.

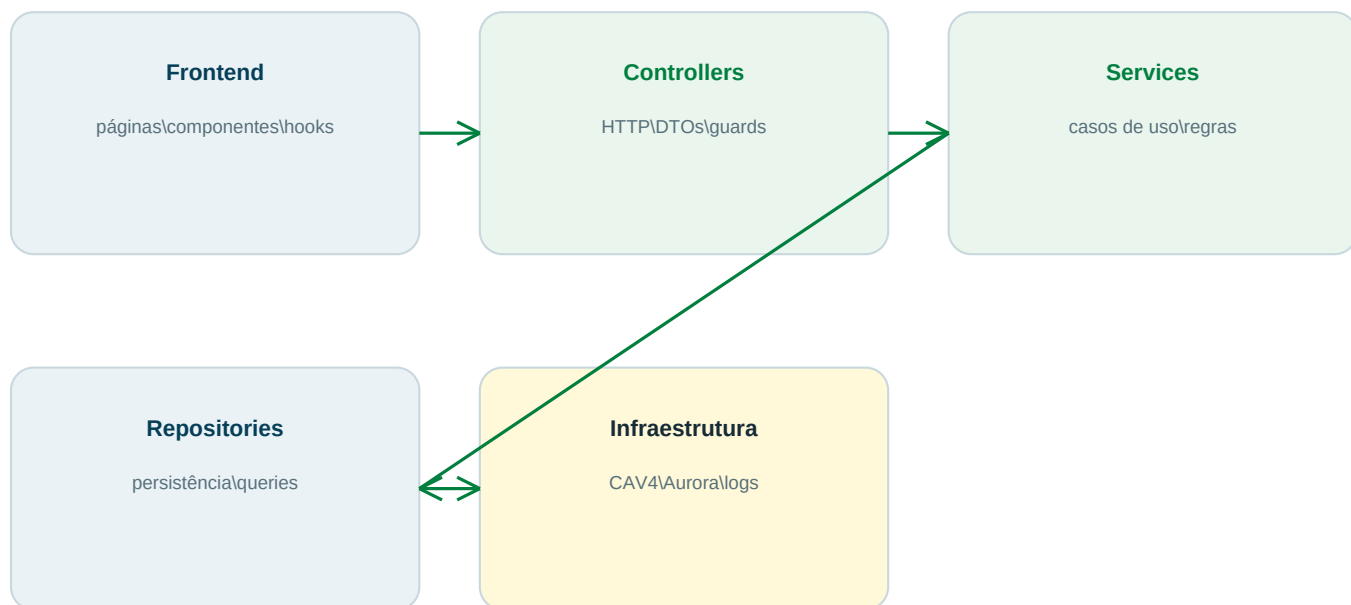
**Fora do escopo da primeira fase:** criação, exclusão ou renomeação de pastas; upload, download, edição ou exclusão de arquivos; alteração física de permissões no servidor de armazenamento.

# 2. Estilo arquitetural

O sistema utiliza uma **arquitetura em camadas com organização modular por domínio** (Layered/Modular Architecture). A separação evita que transporte HTTP, regras de negócio, persistência e integrações externas fiquem misturados.

| Camada         | Responsabilidade                            | Tecnologia/implementação            |
|----------------|---------------------------------------------|-------------------------------------|
| Apresentação   | Rotas HTTP, validação de entrada e resposta | FastAPI routers; Next.js App Router |
| Aplicação      | Casos de uso e orquestração                 | Services por módulo                 |
| Domínio        | Entidades, perfis, regras e autorização     | Modelos, schemas e guards           |
| Persistência   | Consultas e transações                      | Repositories; SQLAlchemy async      |
| Infraestrutura | CAV4, Aurora, logs, configuração            | httpx, PostgreSQL/asyncpg, Alembic  |

## Separação de responsabilidades



### 3. Arquitetura do frontend

O frontend utiliza Next.js com App Router, componentes React reutilizáveis e uma camada de cliente HTTP centralizada. O layout autenticado protege as páginas da aplicação e compõe sidebar, topbar, conteúdo e rodapé.

| Elemento             | Função                                                   |
|----------------------|----------------------------------------------------------|
| app/(app)/layout.tsx | Layout autenticado, sessão e composição visual           |
| app/(app)/dashboard  | Visão executiva e indicadores                            |
| app/(app)/projetos   | Lista, cadastro, detalhe, membros e explorador de pastas |
| app/(app)/relatorios | Consulta e exportação de relatórios                      |
| app/(app)/logs       | Consulta da trilha de auditoria                          |
| components/          | Componentes visuais, guards, formulários e navegação     |
| lib/api-client.ts    | Contrato HTTP único com o backend                        |
| lib/types.ts         | Tipos de domínio consumidos pelo frontend                |

Princípio adotado: o frontend controla apresentação, experiência e visibilidade inicial dos menus, mas a autorização efetiva deve sempre ser aplicada no backend. Ocultar um menu não substitui uma verificação de permissão na API.

### 4. Arquitetura do backend

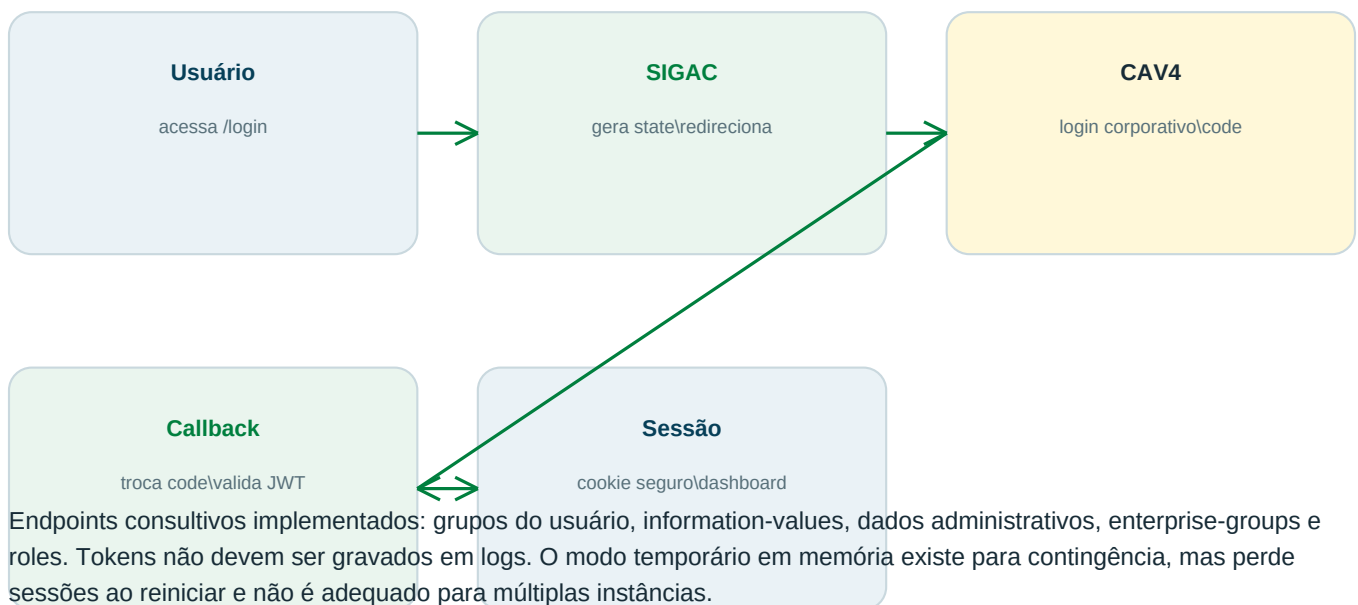
O backend é uma aplicação FastAPI modular. Cada domínio possui controllers, services, repositories, schemas e modelos quando necessário. O acesso ao banco deve ficar encapsulado nos repositories; services não devem depender de detalhes de HTTP.

| Módulo           | Responsabilidade principal                                       |
|------------------|------------------------------------------------------------------|
| api/routes       | Autenticação CAV4, diretório CAV4, health e composição das rotas |
| modules/users    | Usuários, perfis e identidade local                              |
| modules/projects | Projetos, participantes, estados e acesso                        |
| modules/files    | Pastas e metadados consultivos                                   |
| modules/catalogs | Áreas, perfis, módulos, permissões e catálogos                   |
| modules/audit    | Registro e consulta da auditoria                                 |
| core             | Configuração, autorização, logs, CAV4 e sessões                  |
| db               | Engine, modelos SQLAlchemy, seed e base declarativa              |

### 5. Integração e autenticação CAV4

O fluxo segue Authorization Code/OIDC: primeiro o backend valida se já existe uma sessão CAV4 válida; nesse caso, resolve o perfil e redireciona diretamente para /dashboard. Sem sessão válida, o SIGAC inicia o login, o CAV4 autentica o usuário, o callback troca o código por tokens, valida o JWT e consulta os endpoints corporativos quando necessário. O backend então cria a sessão e redireciona o navegador para o frontend.

## Fluxo de autenticação



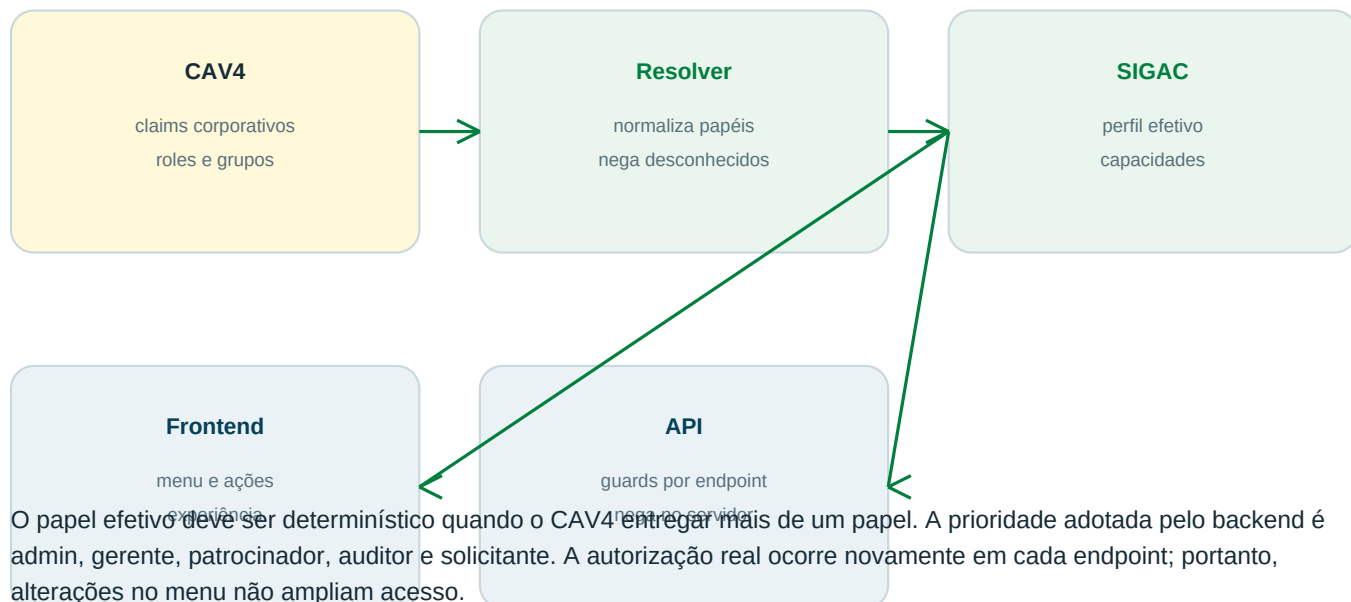
## 6. Autorização e perfis

| Perfil        | Responsabilidade e acesso esperado                                                        |
|---------------|-------------------------------------------------------------------------------------------|
| Administrador | Governança total: usuários, perfis, menus, permissões, projetos, solicitações e auditoria |
| Gerente       | Gestão operacional dos projetos sob sua responsabilidade e consulta de pastas/permissões  |
| Patrocinador  | Acompanhamento executivo e aprovação conforme alçada definida                             |
| Auditor       | Consulta de projetos, pastas e trilha de auditoria; sem alteração operacional             |
| Solicitante   | Solicitação de acesso e consulta limitada ao escopo concedido                             |

A fonte futura dos perfis é o CAV4: grupos, roles e information-values devem ser mapeados para os perfis SIGAC em uma regra formal aprovada. Usuários fixos no seed são apenas bootstrap técnico e não devem substituir a identidade corporativa em produção.

## 6.1. Cadeia de decisão de acesso

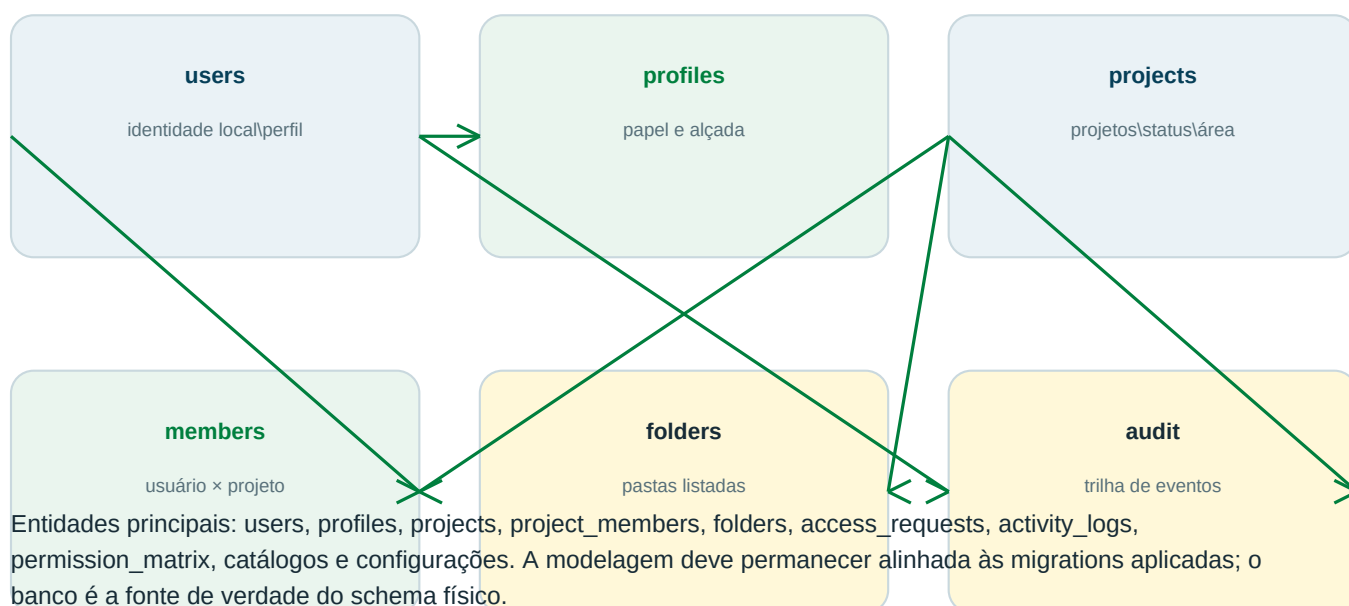
### CAV4 como autoridade



## 7. Modelagem e persistência

O modelo relacional utiliza PostgreSQL Aurora, chaves técnicas UUID, relacionamentos por chaves estrangeiras, índices para consultas frequentes e migrations versionadas pelo Alembic. O ORM SQLAlchemy representa a persistência; schemas Pydantic representam contratos de entrada e saída.

### Modelo lógico resumido



## 8. Pastas e permissões na primeira fase

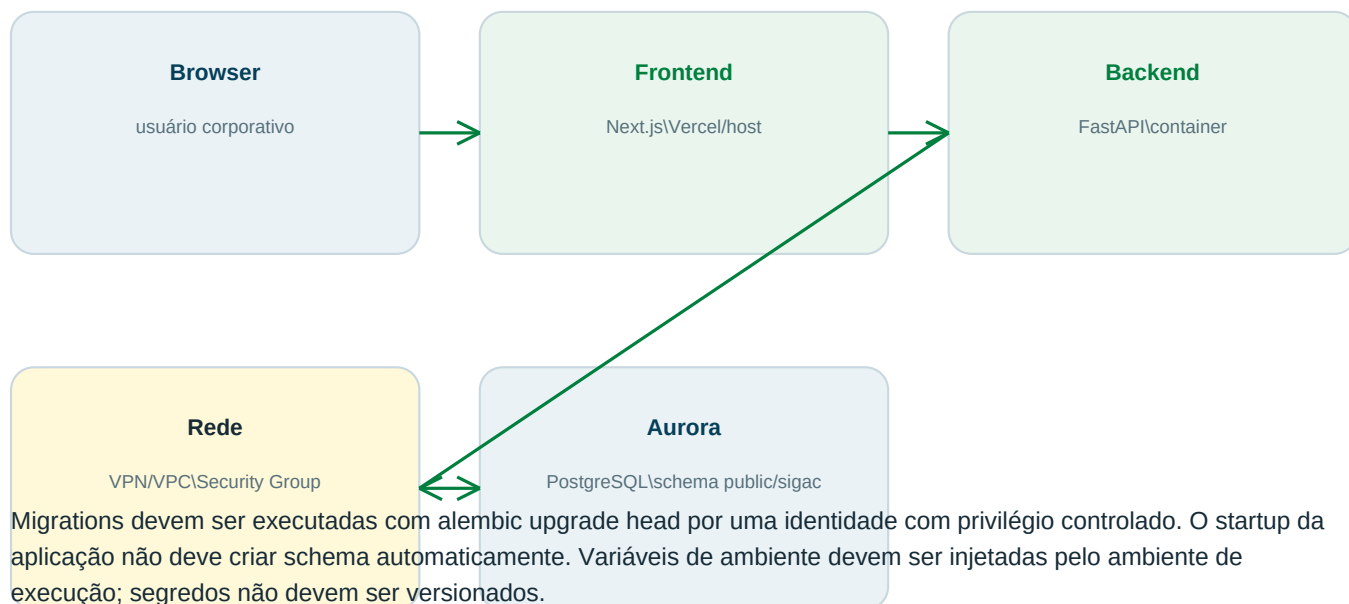
As pastas são listadas do servidor corporativo por integração. O SIGAC apresenta nome, caminho/identificador, projeto, grupos, usuários e nível de acesso quando fornecidos. Leitura e escrita são exibidas como informação de autorização; nenhuma operação física de escrita é executada pelo SIGAC nesta fase.

## 9. Segurança, auditoria e observabilidade

- CORS restrito às origens aprovadas; cookies e sessão com atributos seguros.
- Queries parametrizadas e validação de entrada.
- Logs estruturados/legíveis com request\_id, método, rota, status e duração.
- Nunca registrar senha, client secret, access token ou conteúdo sensível do CAV4.
- Auditar login, logout, alterações de projeto, membros, solicitações, permissões e consultas administrativas.
- Health check deve diferenciar aplicação disponível de dependência de banco indisponível.

## 10. Deploy e operação

### Topologia operacional



## 11. Decisões, riscos e melhorias

| Tema              | Estado atual                  | Próximo passo recomendado                                          |
|-------------------|-------------------------------|--------------------------------------------------------------------|
| Sessão temporária | Contingência em memória       | Persistir sessão/token de forma segura e remover o modo temporário |
| CAV4              | OIDC e consultas de diretório | Formalizar mapeamento de claims/roles para perfis SIGAC            |
| Pastas            | Consulta/listagem             | Definir contrato oficial do servidor e cache somente se aprovado   |
| ORM               | SQLAlchemy + migrations       | Testar metadata contra schema PostgreSQL em CI                     |
| Permissões        | Matriz por perfil e guards    | Centralizar policy engine e testes por endpoint                    |
| Observabilidade   | Logs com request_id           | Centralizar logs, métricas e alertas operacionais                  |

## 12. Critérios de aceite arquitetural

- O gestor aprova o escopo da primeira fase e confirma que não haverá operações de arquivos/pastas.
- Infraestrutura confirma conectividade VPN/VPC, Security Group, DNS e porta 5432.
- DBA aprova schema, grants e execução das migrations.
- Segurança aprova fluxo CAV4, cookies, CORS, logs e retenção de auditoria.
- Produto aprova matriz de perfis, menus e permissões.
- QA valida contratos frontend/backend, autenticação, autorização e principais fluxos.

Documento preparado a partir da implementação atual do frontend, backend, migrations, modelos, componentes e regras de negócio do SIGAC. Deve ser validado pelo gestor antes de ser considerado baseline oficial.